# Lab Assignment 14.5

| | |
|---|---|
| **Program** | **: B. Tech (CSE)** |
| **Course Title** | **: AI Assisted Coding** |
| **Course Code** | **: 23CS002PC304** |
| **Semester** | **: III** |
| **Academic Session** | **: 2025-2026** |
| **Name of Student** | **: Sruthi** |
| **Enrollment No.** | **: 2403A51L10** |
| **Batch No.** | **: 51** |
| **Date** | **: 13-03-2026** |

## Task 1:

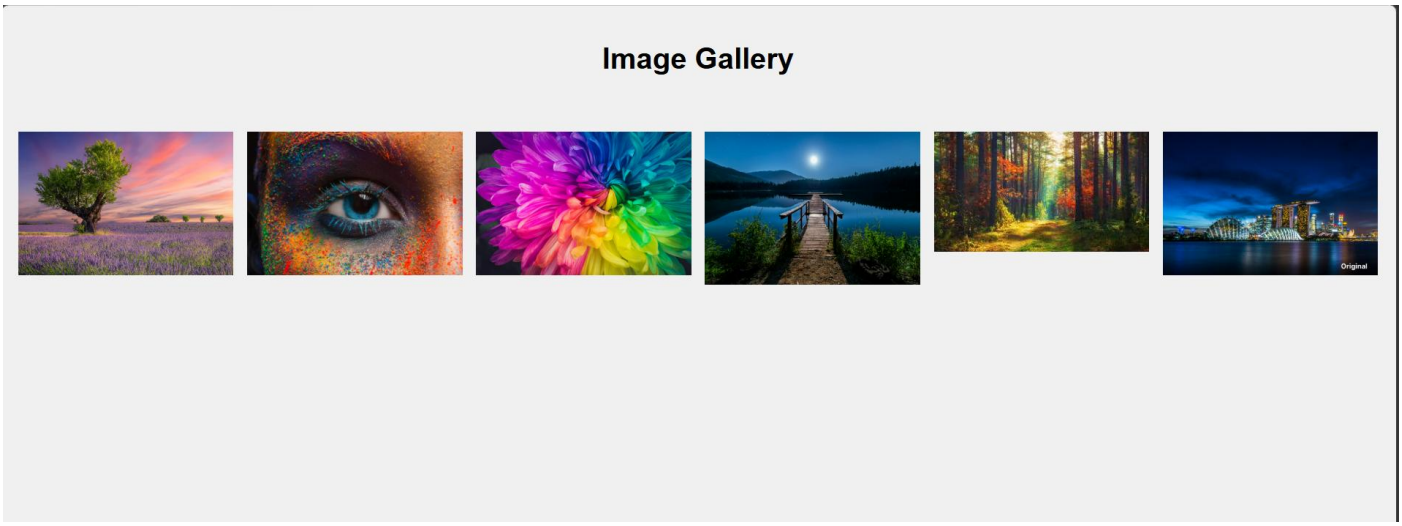
**Prompt:** Create an image gallery web page that display images in a grid layout and optimize layout using CSS Grid and add hover zoom or overlay effects and ensure mobile responsiveness.

## Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 15px;
            padding: 20px;
        }
        .gallery-item {
            position: relative;
            overflow: hidden;
        }
        .gallery-item img {
            width: 100%;
            height: auto;
            transition: transform 0.3s ease;
        }
        .gallery-item:hover img {
            transform: scale(1.1);
        }
        .overlay {
            position: absolute;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            background-color: rgba(0, 0, 0, 0.5);
            color: white;
            display: flex;
            align-items: center;
            justify-content: center;
            opacity: 0;
            transition: opacity 0.3s ease;
        }
```

```html
        .gallery-item:hover .overlay {
            opacity: 1;
        }
    </style>
</head>
<body>
    <h1 style="text-align: center; padding: 20px;">Image Gallery</h1>
    <div class="gallery">
        <div class="gallery-item">
            <img src="https://img.freepik.com/free-photo/lavender-field-sunset-near-valensole_268835-3910.jpg?semt=ais_rp_progressive&w=740&q=80" alt="Image 1">
            <div class="overlay">Image 1</div>
        </div>
        <div class="gallery-item">
            <img src="https://media.istockphoto.com/id/814423752/photo/eye-of-model-with-colorful-art-make-up-close-up.jpg?s=612x612&w=0&k=20&c=115OdMWjgCKycMMShP8UK94ELVlEGvt7GmB_esHWPYE=" alt="Image 2">
            <div class="overlay">Image 2</div>
        </div>
        <div class="gallery-item">
            <img src="https://media.istockphoto.com/id/520700958/photo/beautiful-flowers-background.jpg?s=612x612&w=0&k=20&c=A7WF8MScj3YNBTA-qFQEKm-Xphzmi_mfaOqjq27--j4=" alt="Image 3">
            <div class="overlay">Image 3</div>
        </div>
        <div class="gallery-item">
            <img src="https://images.pexels.com/photos/414612/pexels-photo-414612.jpeg?cs=srgb&dl=pexels-souvenirpixels-414612.jpg&fm=jpg" alt="Image 4">
            <div class="overlay">Image 4</div>
        </div>
        <div class="gallery-item">
            <img src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcR7UGj1d3j2PMKdQg0Q_ki1pZGpmqhPq53kSA&s" alt="Image 5">
            <div class="overlay">Image 5</div>
        </div>
        <div class="gallery-item">
            <img src="https://ik.imagekit.io/serenity/ByteofDev/Blog_Content_Images/tr:f-jpg/speed_up_images_original_R8oje17ew" alt="Image 6">
            <div class="overlay">Image 6</div>
        </div>
    </div>
    </body>
</html>
```

## Output:




## Explanation:

Creates a simple image gallery using HTML and CSS. The gallery is displayed in a grid layout, and each image has a hover effect that enlarges the image and shows an overlay with the image's name. The gallery is responsive, meaning it will adjust the number of columns based on the screen size.

The CSS styles define the layout and appearance of the gallery, including the hover effects for the images and overlays. The images are sourced from various URLs, and each image is wrapped in a div with the class "gallery-item" to apply the necessary styles.

The gallery will automatically adjust to fit the screen size, making it suitable for both desktop and mobile devices.

# Task 2:

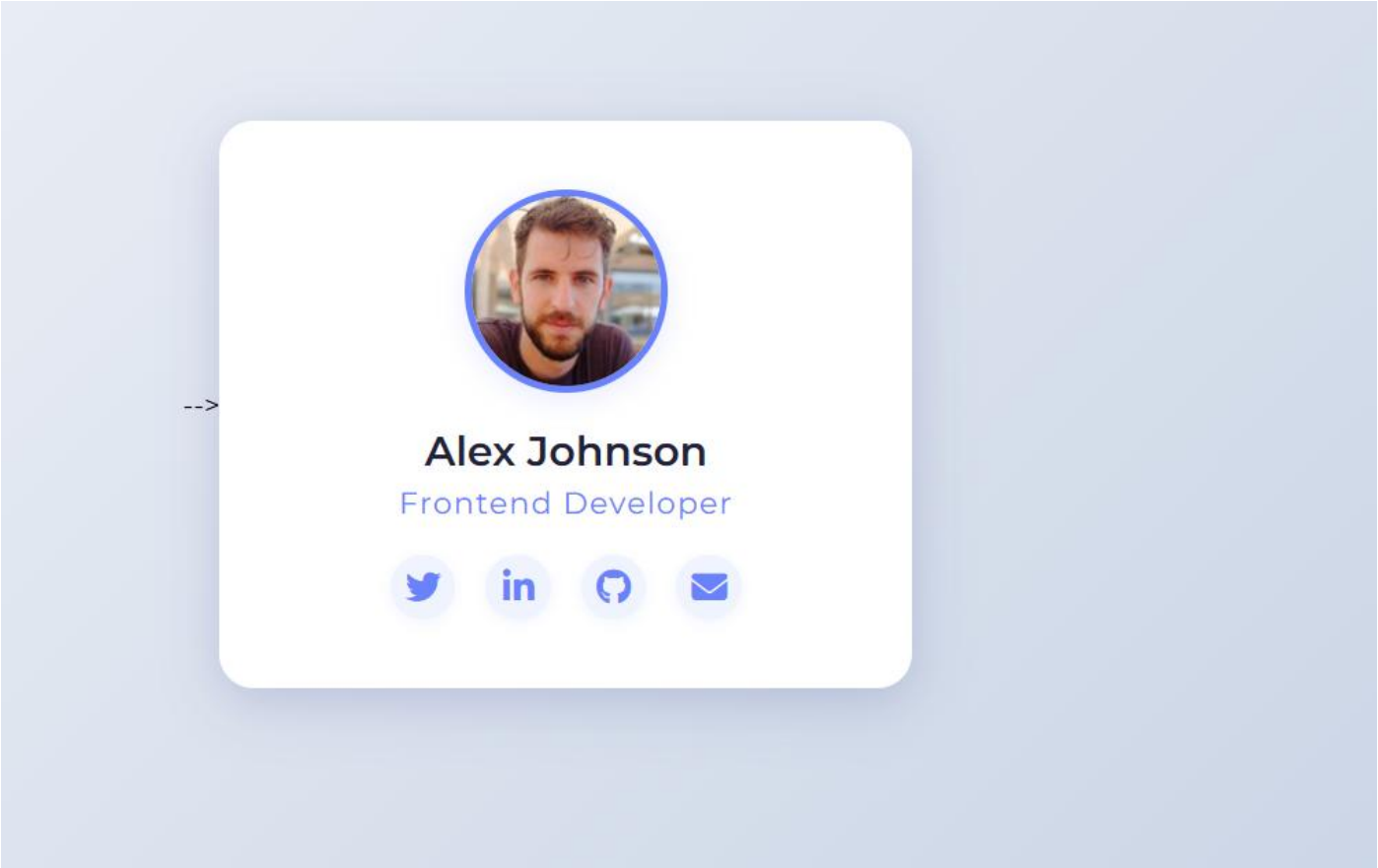
**Prompt:** Create a profile card web page that display user image, name, role, and social media links and Use modern UI colors and fonts and design the card using Flexbox and add hover animation and shadow effects.

# Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <link href="https://fonts.googleapis.com/css?family=Montserrat:600,400&display=swap" rel="stylesheet">
    <style>
        body {
            background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
            min-height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
            font-family: 'Montserrat', Arial, sans-serif;
        }
        .profile-card {
            background: #fff;
            border-radius: 20px;
            box-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.18);
            padding: 2.5rem 2rem;
            max-width: 340px;
            width: 100%;
            display: flex;
            flex-direction: column;
            align-items: center;
            transition: transform 0.3s, box-shadow 0.3s;
        }
        .profile-card:hover {
            transform: translateY(-10px) scale(1.03);
            box-shadow: 0 16px 40px 0 rgba(31, 38, 135, 0.22);
        }
        .profile-img {
            width: 110px;
            height: 110px;
            border-radius: 50%;
            object-fit: cover;
            border: 4px solid #6a82fb;
            margin-bottom: 1.2rem;
            box-shadow: 0 4px 16px rgba(106,130,251,0.15);
        }
        .profile-name {
            font-size: 1.5rem;
            font-weight: 600;
            color: #22223b;
            margin-bottom: 0.3rem;
        }
        .profile-role {
            font-size: 1.1rem;
            color: #6a82fb;
            margin-bottom: 1.2rem;
            letter-spacing: 1px;
        }
        .social-links {
            display: flex;
            gap: 1.1rem;
        }
        .social-links a {
            display: flex;
            align-items: center;
            justify-content: center;
            width: 38px;
            height: 38px;
            border-radius: 50%;
            background: #f0f4ff;
            color: #6a82fb;
            font-size: 1.3rem;
            transition: background 0.2s, color 0.2s, box-shadow 0.2s;
            box-shadow: 0 2px 8px rgba(106,130,251,0.08);
            text-decoration: none;
        }
        .social-links a:hover {
            background: #6a82fb;
            color: #fff;
            box-shadow: 0 4px 16px rgba(106,130,251,0.18);
        }
    </style>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.2/css/all.min.css">
</head>
<body>
    <div class="profile-card">
        <img class="profile-img" src="https://randomuser.me/api/portraits/men/32.jpg" alt="User Image">
        <div class="profile-name">Alex Johnson</div>
        <div class="profile-role">Frontend Developer</div>
        <div class="social-links">
            <a href="https://twitter.com/" target="_blank" title="Twitter"><i class="fab fa-twitter"></i></a>
```

```html
        <div class="profile-name">Alex Johnson</div>
        <div class="profile-role">Frontend Developer</div>
        <div class="social-links">
            <a href="https://twitter.com/" target="_blank" title="Twitter"><i class="fab fa-twitter"></i></a>
            <a href="https://linkedin.com/" target="_blank" title="LinkedIn"><i class="fab fa-linkedin-in"></i></a>
            <a href="https://github.com/" target="_blank" title="GitHub"><i class="fab fa-github"></i></a>
            <a href="mailto:alex@email.com" title="Email"><i class="fas fa-envelope"></i></a>
        </div>
    </div>
</body>
</html>
```

## Output:




## Explanation:

This HTML code creates a stylish profile card using modern CSS techniques. The card features a circular profile image, the user's name, their role, and social media links. The design includes hover effects that enhance interactivity, such as scaling the card and changing the background colour of social media icons.

# Task 3:

**Prompt:** Develop a dynamic shopping cart system. That add, remove, and update product quantities and display real-time total price. Use JavaScript objects to manage cart state and Persist cart data in local Storage.

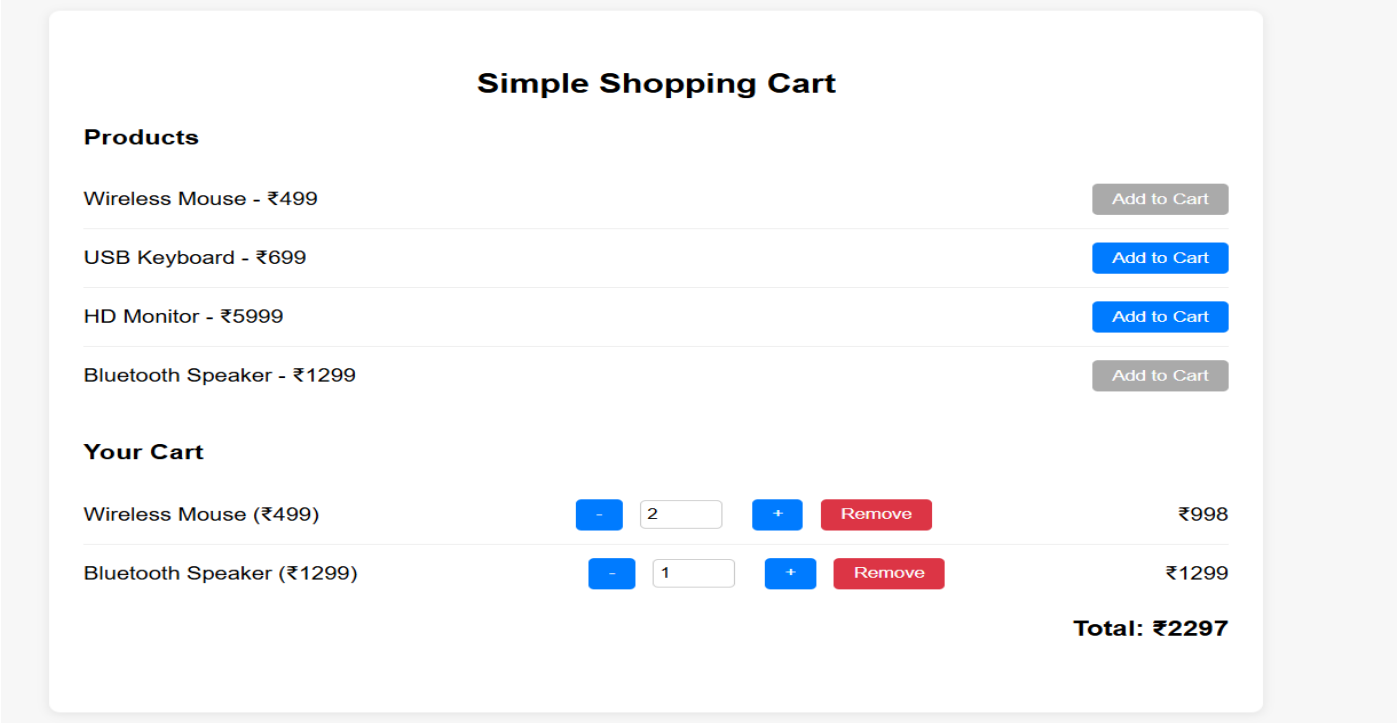
## Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Dynamic Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f7f7f7;
            margin: 0;
            padding: 0;
        }
        .container {
            max-width: 800px;
            margin: 40px auto;
            background: #fff;
            border-radius: 8px;
            box-shadow: 0 2px 8px rgba(0,0,0,0.08);
            padding: 32px 24px;
        }
        h2 {
            text-align: center;
            margin-bottom: 24px;
        }
        .products, .cart {
            margin-bottom: 32px;
        }
        .product, .cart-item {
            display: flex;
            align-items: center;
            justify-content: space-between;
            padding: 12px 0;
            border-bottom: 1px solid #eee;
        }
        .product:last-child, .cart-item:last-child {
            border-bottom: none;
        }
        .product button, .cart-item button {
            background: #007bff;
            color: #fff;
            border: none;
            padding: 6px 14px;
            border-radius: 4px;
            cursor: pointer;
            margin-left: 8px;
        }
        .product button:disabled {
            background: #aaa;
            cursor: not-allowed;
        }
        .cart-item input[type="number"] {
            width: 48px;
            padding: 4px;
            margin: 0 8px;
            border: 1px solid #ccc;
            border-radius: 4px;
        }
        .total {
            text-align: right;
            font-size: 1.2em;
            font-weight: bold;
            margin-top: 12px;
        }
        .empty-cart {
            text-align: center;
            color: #888;
            margin: 24px 0;
```

```
        }
    </style>
</head>
<body>
<div class="container">
    <h2>Simple Shopping Cart</h2>
    <div class="products">
        <h3>Products</h3>
        <div id="product-list"></div>
    </div>
    <div class="cart">
        <h3>Your Cart</h3>
        <div id="cart-list"></div>
        <div class="total" id="cart-total"></div>
    </div>
</div>
<script>
const products = [
    { id: 1, name: "Wireless Mouse", price: 499 },
    { id: 2, name: "USB Keyboard", price: 699 },
    { id: 3, name: "HD Monitor", price: 5999 },
    { id: 4, name: "Bluetooth Speaker", price: 1299 }
];

let cart = {};

function saveCart() {
    localStorage.setItem('cart', JSON.stringify(cart));
}
function loadCart() {
    const data = localStorage.getItem('cart');
    cart = data ? JSON.parse(data) : {};
}
function renderProducts() {
    const list = document.getElementById('product-list');
    list.innerHTML = '';
    products.forEach(prod => {
        const btnDisabled = cart[prod.id] ? 'disabled' : '';
        list.innerHTML += `
            <div class="product">
                <span>${prod.name} - ₹${prod.price}</span>
                <button ${btnDisabled} onclick="addToCart(${prod.id})">Add to Cart</button>
            </div>
        `;
    });
}
function renderCart() {
    const list = document.getElementById('cart-list');
    const totalDiv = document.getElementById('cart-total');
    let total = 0;
    list.innerHTML = '';
    const ids = Object.keys(cart);
    if (ids.length === 0) {
        list.innerHTML = `<div class="empty-cart">Your cart is empty.</div>`;
        totalDiv.textContent = '';
        return;
    }
    ids.forEach(id => {
        const prod = products.find(p => p.id == id);
        const qty = cart[id];
        const subtotal = prod.price * qty;
        total += subtotal;
```

```
        list.innerHTML += `
            <div class="cart-item">
                <span>${prod.name} (₹${prod.price})</span>
                <div>
                    <button onclick="updateQty(${id}, -1)">-</button>
                    <input type="number" min="1" value="${qty}" onchange="setQty(${id}, this.value)">
                    <button onclick="updateQty(${id}, 1)">+</button>
                    <button onclick="removeFromCart(${id})" style="background:#dc3545;">Remove</button>
                </div>
                <span>₹${subtotal}</span>
            </div>
        `;
    });
    totalDiv.textContent = `Total: ₹${total}`;
}
window.addToCart = function(id) {
    if (!cart[id]) cart[id] = 1;
    saveCart();
    renderProducts();
    renderCart();
};
window.removeFromCart = function(id) {
    delete cart[id];
    saveCart();
    renderProducts();
    renderCart();
};
window.updateQty = function(id, delta) {
    if (!cart[id]) return;
    let newQty = cart[id] + delta;
    if (newQty < 1) newQty = 1;
    cart[id] = newQty;
    saveCart();
    renderCart();
};
window.setQty = function(id, value) {
    let qty = parseInt(value);
    if (isNaN(qty) || qty < 1) qty = 1;
    cart[id] = qty;
    saveCart();
    renderCart();
};
loadCart();
renderProducts();
renderCart();
</script>
</body>
</html>
```

# Output:



## Explanation:

This code creates a simple shopping cart application using HTML, CSS, and JavaScript. It allows users to add products to their cart, update quantities, and remove items. The cart's state is saved in the browser's local storage, so it persists across page reloads. The UI is designed to be clean and user-friendly, with clear sections for products and the shopping cart.

The JavaScript code manages the cart's functionality, including adding items, updating quantities, and calculating the total price. The application is responsive and provides feedback to the user when actions are taken, such as disabling the "Add to Cart" button for items already in the cart.

# Task 4:

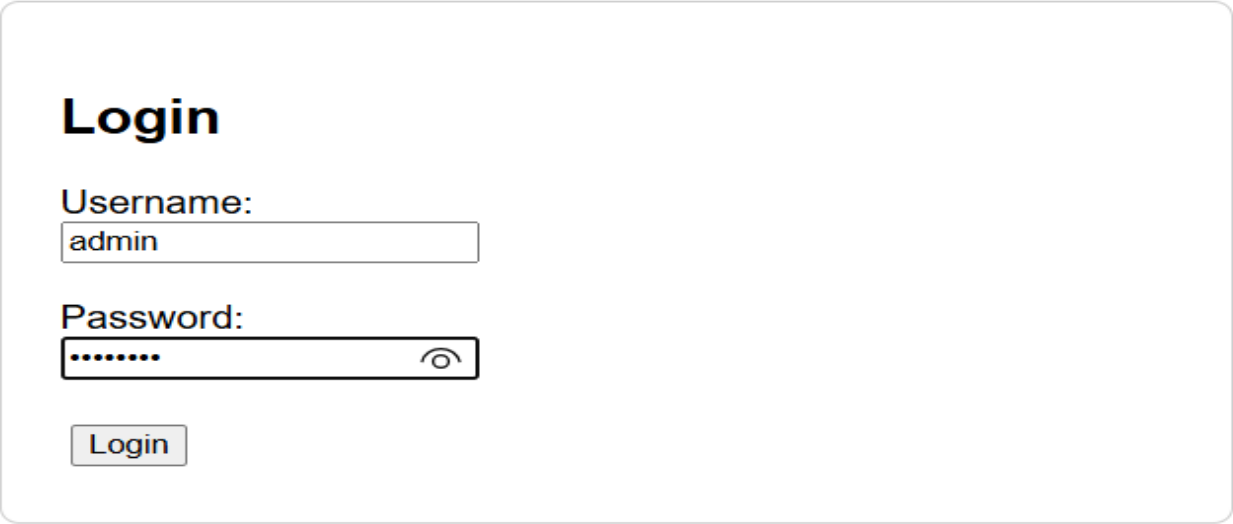
**Prompt:** Create a role-based web application that as Admin, Editor, User roles and conditional UI rendering and Design access control logic and maintain secure state transitions and Create scalable UI components.

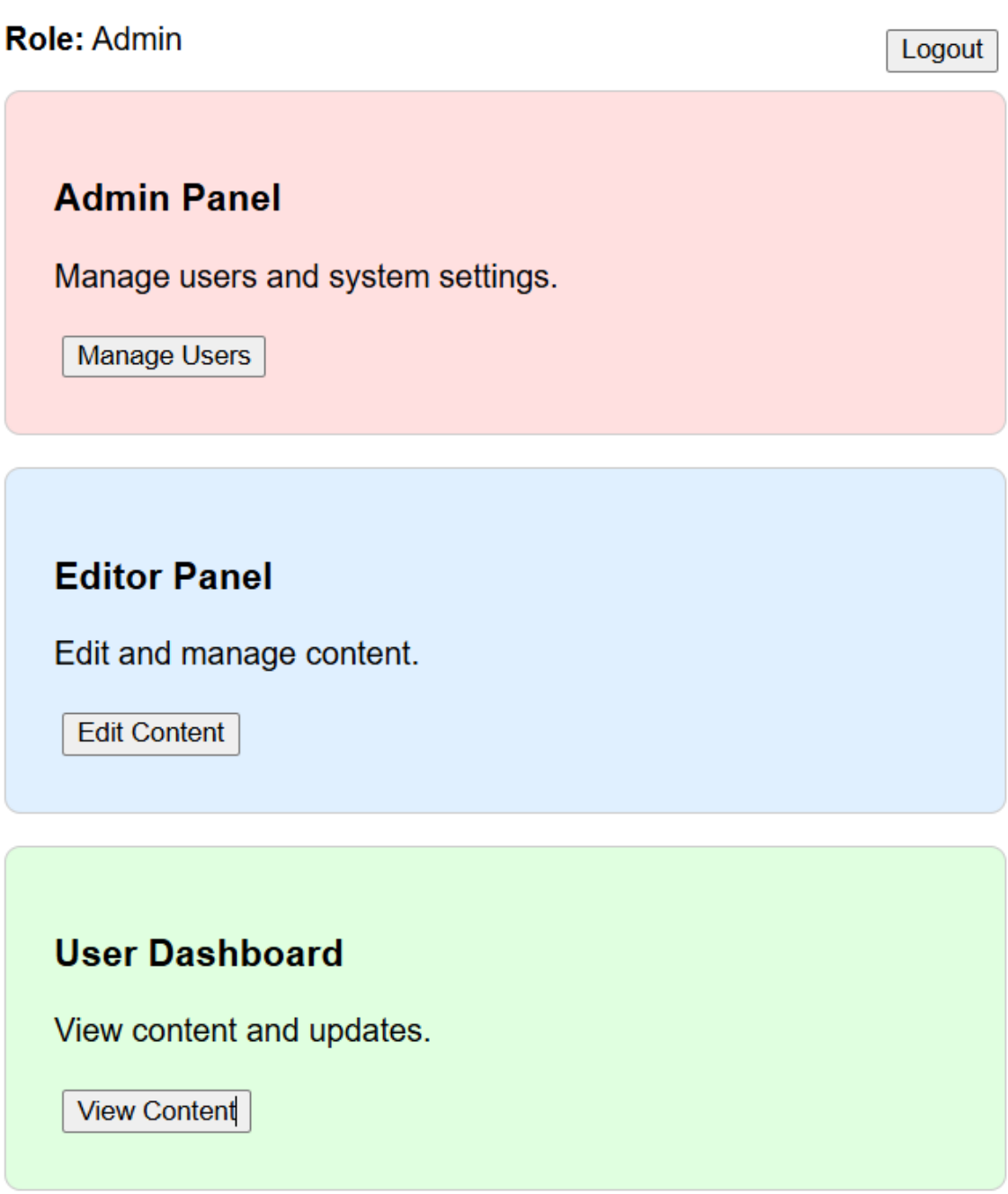
## Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Role-Based Web Application</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; }
        .container { max-width: 500px; margin: auto; }
        .card { border: 1px solid #ccc; border-radius: 8px; padding: 24px; margin-bottom: 16px; }
        .admin { background: #ffe0e0; }
        .editor { background: #e0f0ff; }
        .user { background: #e0ffe0; }
        button { margin: 4px; }
        .logout { float: right; }
    </style>
</head>
<body>
    <div class="container" id="app"></div>
    <script>
        // Roles and permissions
        const ROLES = ['Admin', 'Editor', 'User'];
        const PERMISSIONS = {
            Admin: ['manage_users', 'edit_content', 'view_content'],
            Editor: ['edit_content', 'view_content'],
            User: ['view_content']
        };

        // Simulated users
        const USERS = [
            { username: 'admin', password: 'admin123', role: 'Admin' },
            { username: 'editor', password: 'editor123', role: 'Editor' },
            { username: 'user', password: 'user123', role: 'User' }
        ];

        // Secure state management
        let currentUser = null;

        // UI Components
        function LoginForm() {
            return `
                <div class="card">
                    <h2>Login</h2>
                    <form id="loginForm">
                        <label>Username:<br><input type="text" name="username" required></label><br><br>
                        <label>Password:<br><input type="password" name="password" required></label><br><br>
                        <button type="submit">Login</button>
                    </form>
                </div>
            `;
        }
        function Navbar(user) {
            return `
                <div style="margin-bottom:16px;">
                    <strong>Role:</strong> ${user.role}
                    <button class="logout" id="logoutBtn">Logout</button>
```

```
            </div>
        `;
    }

    function AdminPanel() {
        return `
            <div class="card admin">
                <h3>Admin Panel</h3>
                <p>Manage users and system settings.</p>
                <button onclick="alert('User management accessed!')">Manage Users</button>
            </div>
        `;
    }

    function EditorPanel() {
        return `
            <div class="card editor">
                <h3>Editor Panel</h3>
                <p>Edit and manage content.</p>
                <button onclick="alert('Content editor accessed!')">Edit Content</button>
            </div>
        `;
    }

    function UserPanel() {
        return `
            <div class="card user">
                <h3>User Dashboard</h3>
                <p>View content and updates.</p>
                <button onclick="alert('Viewing content!')">View Content</button>
            </div>
        `;
    }

    // Access control logic
    function hasPermission(role, permission) {
        return PERMISSIONS[role] && PERMISSIONS[role].includes(permission);
    }

    // Main render function
    function render() {
        const app = document.getElementById('app');
        if (!currentUser) {
            app.innerHTML = LoginForm();
            document.getElementById('loginForm').onsubmit = function(e) {
                e.preventDefault();
                const username = this.username.value;
                const password = this.password.value;
                const user = USERS.find(u => u.username === username && u.password === password);
                if (user) {
                    currentUser = { ...user }; // Secure state transition
                    render();
                } else {
                    alert('Invalid credentials!');
                }
            };
        } else {
            let panels = '';
            if (hasPermission(currentUser.role, 'manage_users')) panels += AdminPanel();
            if (hasPermission(currentUser.role, 'edit_content')) panels += EditorPanel();
            if (hasPermission(currentUser.role, 'view_content')) panels += UserPanel();

            app.innerHTML = Navbar(currentUser) + panels;
            document.getElementById('logoutBtn').onclick = function() {
                currentUser = null; // Secure state transition
                render();
            };
        }
    }

    // Initial render
    render();
</script>
</body>
</html>
```

**Output:**


Login
Username:
admin
Password:
Login


Role: Admin
Logout
Admin Panel
Manage users and system settings.
Manage Users
Editor Panel
Edit and manage content.
Edit Content
User Dashboard
View content and updates.
View Content

## Explanation:

This code implements a simple role-based web application using HTML, CSS, and JavaScript. It defines three user roles: Admin, Editor, and User, each with specific permissions. The application includes a login form, a navigation bar displaying the user's role, and different panels for each role. The access control logic ensures that users can only access features corresponding to their roles. The state management is handled securely by maintaining the current user in a variable and updating the UI accordingly.

# Task 5:

**Prompt:** Design a multilingual website that Switch languages dynamically and persist language preference and structure translation files and it handle RTL/LTR layouts and improve accessibility.

## Code:

```html
<!DOCTYPE html>
<html lang="en" dir="ltr">
<head>
    <meta charset="UTF-8">
    <title>Multilingual Website</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body[dir="rtl"] {
            direction: rtl;
            text-align: right;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }
        body[dir="ltr"] {
            direction: ltr;
            text-align: left;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }
        .lang-switcher {
            margin: 1em 0;
        }
        .lang-switcher button {
            margin-right: 0.5em;
            padding: 0.5em 1em;
            cursor: pointer;
        }
        main {
            max-width: 600px;
            margin: auto;
        }
        /* Accessibility: focus styles */
        button:focus, a:focus {
            outline: 2px solid #0078D4;
            outline-offset: 2px;
        }
    </style>
</head>
<body>
    <nav aria-label="Language Switcher" class="lang-switcher">
        <button type="button" data-lang="en" aria-label="Switch to English">English</button>
        <button type="button" data-lang="es" aria-label="Switch to Spanish">Español</button>
        <button type="button" data-lang="ar" aria-label="Switch to Arabic">العربية</button>
    </nav>
    <main>
        <h1 id="title"></h1>
        <p id="welcome"></p>
        <a href="#" id="learnMore"></a>
    </main>
    <script>
        // Translation files structure
        const translations = {
            en: {
                dir: "ltr",
                title: "Welcome to Our Website",
                welcome: "This is a multilingual website. Switch languages using the buttons above.",
                learnMore: "Learn more about accessibility"
```

```
        },
        es: {
            dir: "ltr",
            title: "Bienvenido a Nuestro Sitio Web",
            welcome: "Este es un sitio web multilingüe. Cambia de idioma usando los botones de arriba.",
            learnMore: "Aprende más sobre accesibilidad"
        },
        ar: {
            dir: "rtl",
            title: "مرحبًا بكم في موقعنا",
            welcome: "هذا موقع متعدد اللغات. قم بتغيير اللغة باستخدام الأزرار أعلاه.",
            learnMore: "تعرف على المزيد حول إمكانية الوصول"
        }
    };

    // Accessibility: focus management
    function focusMain() {
        document.querySelector('main').setAttribute('tabindex', '-1');
        document.querySelector('main').focus();
    }
    // Load language from localStorage or default to English
    function getSavedLang() {
        return localStorage.getItem('lang') || 'en';
    }
    function setLang(lang) {
        if (!translations[lang]) lang = 'en';
        localStorage.setItem('lang', lang);
        document.documentElement.lang = lang;
        document.body.setAttribute('dir', translations[lang].dir);

        // Update content
        document.getElementById('title').textContent = translations[lang].title;
        document.getElementById('welcome').textContent = translations[lang].welcome;
        document.getElementById('learnMore').textContent = translations[lang].learnMore;
        document.getElementById('learnMore').setAttribute('href', lang === 'ar' ? 'https://www.w3.org/WAI/standards-guidelines/ar/' : 'https://www.w3.org/WAI/');
    }
    // Event listeners for language switcher
    document.querySelectorAll('.lang-switcher button').forEach(btn => {
        btn.addEventListener('click', e => {
            const lang = btn.getAttribute('data-lang');
            setLang(lang);
            focusMain();
        });
    });
    // Initial load
    setLang(getSavedLang());
</script>
</body>
</html>
```

## Output:

English Español العربية

### Welcome to Our Website

This is a multilingual website. Switch languages using the buttons above.

Learn more about accessibility

## Explanation:

This HTML document is a multilingual website that supports English, Spanish, and Arabic. It includes a language switcher that allows users to change the language of the content dynamically.

# Task 6:

**Prompt:** Create an online exam interface that contains Timer, questions, and submission and prevent multiple submissions (accept only one submission) and manage exam state and improve exam UX and ensure accessibility.

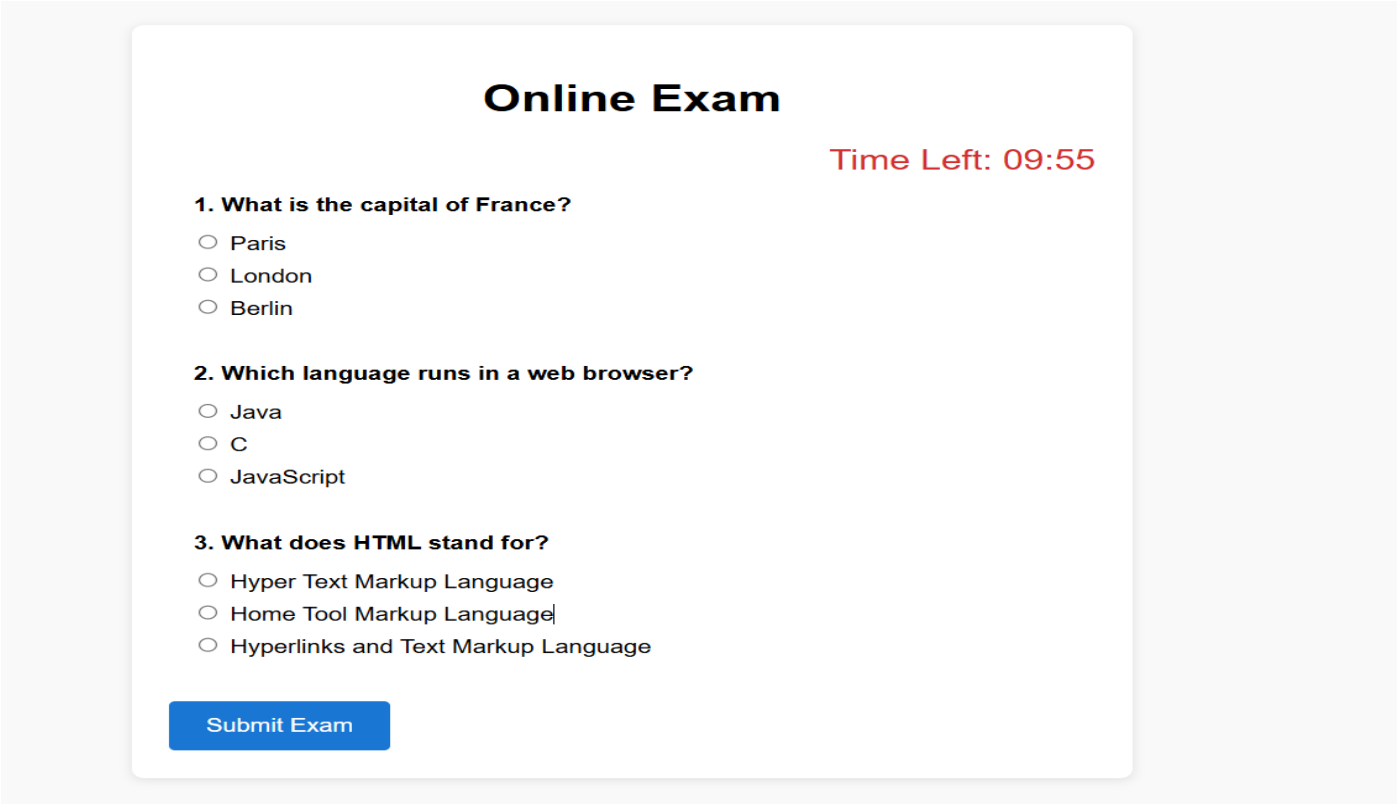
# Code:

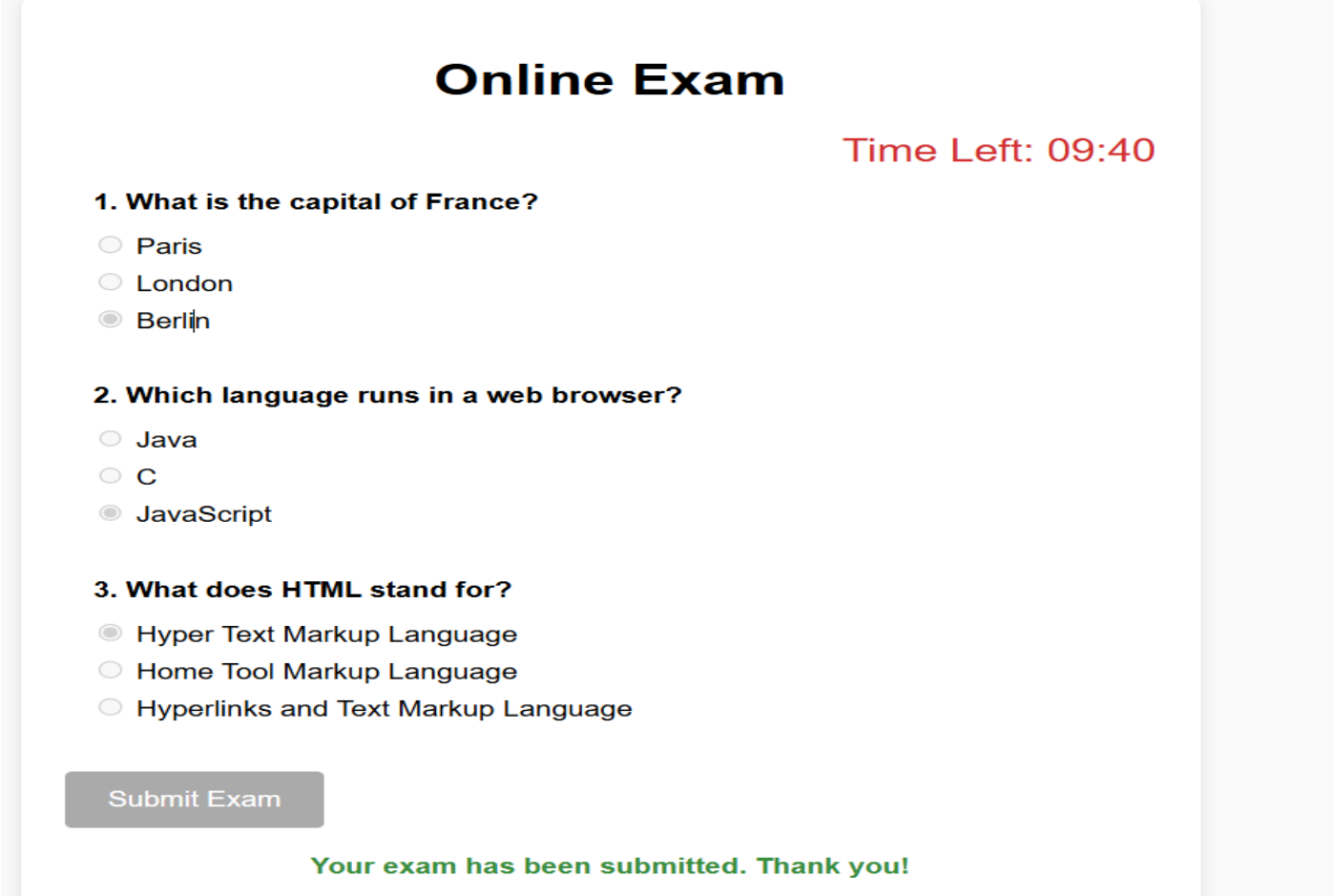
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Online Exam Interface</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body { font-family: Arial, sans-serif; margin: 0; background: #f9f9f9; }
        .container { max-width: 600px; margin: 40px auto; background: #fff; padding: 24px; border-radius: 8px; box-shadow: 0 2px 8px rgba(0,0,0,0.1);}
        h1 { text-align: center; }
        .timer { font-size: 1.5em; color: #d32f2f; text-align: right; margin-bottom: 16px;}
        fieldset { border: none; margin-bottom: 20px; }
        legend { font-weight: bold; }
        label { display: block; margin: 8px 0; }
        button { background: #1976d2; color: #fff; border: none; padding: 12px 24px; border-radius: 4px; font-size: 1em; cursor: pointer;}
        button[disabled] { background: #aaa; cursor: not-allowed;}
        .submitted-msg { color: #388e3c; font-weight: bold; text-align: center; margin-top: 20px;}
        .visually-hidden { position: absolute; left: -10000px; top: auto; width: 1px; height: 1px; overflow: hidden;}
        @media (max-width: 600px) {
            .container { padding: 10px; }
        }
    </style>
</head>
<body>
    <main class="container" aria-labelledby="exam-title">
        <h1 id="exam-title">Online Exam</h1>
        <div class="timer" id="timer" aria-live="polite" aria-atomic="true">Time Left: 10:00</div>
        <form id="examForm" autocomplete="off" aria-describedby="exam-desc">
            <p id="exam-desc" class="visually-hidden">Answer all questions and submit before the timer ends. Only one submission is allowed.</p>
            <fieldset>
                <legend>1. What is the capital of France?</legend>
                <label>
                    <input type="radio" name="q1" value="Paris" required>
                    Paris
                </label>
                <label>
                    <input type="radio" name="q1" value="London">
                    London
                </label>
                <label>
                    <input type="radio" name="q1" value="Berlin">
                    Berlin
                </label>
            </fieldset>
            <fieldset>
                <legend>2. Which language runs in a web browser?</legend>
                <label>
                    <input type="radio" name="q2" value="Java" required>
                    Java
                </label>
                <label>
                    <input type="radio" name="q2" value="C">
                    C
                </label>
                <label>
```

```html
                    <input type="radio" name="q2" value="JavaScript">
                    JavaScript
                </label>
            </fieldset>
            <fieldset>
                <legend>3. What does HTML stand for?</legend>
                <label>
                    <input type="radio" name="q3" value="Hyper Text Markup Language" required>
                    Hyper Text Markup Language
                </label>
                <label>
                    <input type="radio" name="q3" value="Home Tool Markup Language">
                    Home Tool Markup Language
                </label>
                <label>
                    <input type="radio" name="q3" value="Hyperlinks and Text Markup Language">
                    Hyperlinks and Text Markup Language
                </label>
            </fieldset>
            <button type="submit" id="submitBtn">Submit Exam</button>
        </form>
        <div id="submittedMsg" class="submitted-msg" role="status" aria-live="polite" style="display:none;">
            Your exam has been submitted. Thank you!
        </div>
    </main>
    <script>
        // Exam State
        let submitted = false;
        let timeLeft = 600; // 10 minutes in seconds
        const timerEl = document.getElementById('timer');
        const form = document.getElementById('examForm');
        const submitBtn = document.getElementById('submitBtn');
        const submittedMsg = document.getElementById('submittedMsg');

        // Accessibility: focus management
        function focusFirstInput() {
            const firstInput = form.querySelector('input[type="radio"]');
            if (firstInput) firstInput.focus();
        }

        // Timer logic
        function updateTimer() {
            const min = String(Math.floor(timeLeft / 60)).padStart(2, '0');
            const sec = String(timeLeft % 60).padStart(2, '0');
            timerEl.textContent = `Time Left: ${min}:${sec}`;
            if (timeLeft <= 0) {
                timerEl.textContent = "Time's up!";
                if (!submitted) {
                    handleSubmit();
                }
```

```
        }
    }

    function startTimer() {
        updateTimer();
        const interval = setInterval(() => {
            if (timeLeft > 0 && !submitted) {
                timeLeft--;
                updateTimer();
            } else {
                clearInterval(interval);
            }
        }, 1000);
    }

    // Prevent multiple submissions
    function handleSubmit(e) {
        if (e) e.preventDefault();
        if (submitted) return;
        submitted = true;
        submitBtn.disabled = true;
        Array.from(form.elements).forEach(el => el.disabled = true);
        submittedMsg.style.display = 'block';
        form.setAttribute('aria-disabled', 'true');
        // Optionally, send data to server here
    }

    form.addEventListener('submit', handleSubmit);

    // Prevent resubmission on reload (optional, for demo)
    window.addEventListener('beforeunload', function (e) {
        if (!submitted && timeLeft > 0) {
            e.preventDefault();
            e.returnValue = '';
        }
    });

    // Accessibility: focus first input on load
    window.onload = function() {
        focusFirstInput();
        startTimer();
    };
  </script>
</body>
</html>
```

# Output:






## Explanation:

This HTML document represents an online exam interface with a timer and multiple-choice questions. The structure includes
a main container with a title, a timer, a form containing the questions, and a submission message. The JavaScript handles the timer countdown, form submission, and prevents multiple submissions.